

# Rapport de Planning et Suivi de Projet - Phase #1 + Phase #2 + Phase #3

**Projet :** Creative-Yumland

**Groupe :** Radouane, Matthieu, Rayene

**Période :** Phase #1 (3 semaines) + Phase #2 (3 semaines) + Phase #3 (2 semaines)

**Date :** Février-Avril 2026

---

## 1. Calendrier de réalisation (Planning)

Le travail sur cette première phase s'est étalé sur trois semaines consécutives, suivant une progression logique allant de l'organisation à la production technique.

### Semaine 1 : Initialisation et Brainstorming

- **Mise en place technique :** Création du dépôt de code sur **GitHub**.
- **Organisation :** Mise en place des outils de communication et de gestion :
  - Création d'un serveur **Discord** (réunions vocales et suivi).
  - Groupe **WhatsApp** (communication rapide).
  - Tableau **Trello** (type Kanban) pour le suivi des tâches.
- **Conception :** Réflexions sur le thème du restaurant. Trois pistes ont été évoquées : les composants informatiques (cartes graphiques), la gastronomie française et la cuisine japonaise. Le détail du choix final est disponible dans le *Fichier de Conception*.

### Semaine 2 : Conception Graphique et Développement Autonome

- **Cadrage :** Réunion vocale sur Discord pour valider le nom du restaurant, le logo et la charte graphique (couleurs, polices).
- **Répartition :** Attribution équitable des pages à intégrer (3 pages par membre).
- **Production :** Début du codage en autonomie des structures HTML et des premiers styles CSS.

### Semaine 3 : Mutualisation, Débogage et Finalisation

- **Mise en commun :** Revue de l'avancement global en cours de semaine.
- **Résolution technique :** Centralisation du CSS commun pour éviter les répétitions.

- **Finalisation** : Bilan général le samedi après-midi pour valider le fonctionnement de chaque partie.
- **Livraison** : Rédaction des fichiers PDF (Rapport et Conception), du ReadMe et fusion (merge) des branches sur GitHub le dimanche soir.

## 2. Répartition des tâches

Afin de garantir une charge de travail équitable, les responsabilités ont été divisées comme suit :

| Membre   | Pages HTML & Missions Techniques                                                           | Documentation          |
|----------|--------------------------------------------------------------------------------------------|------------------------|
| Radouane | Page d'accueil, Page de notation, Page de présentation, Barre de navigation (Nav), Footer. | Rapport de projet.     |
| Matthieu | Page de profil, Page de connexion, Page d'inscription.                                     | Fichier ReadMe.        |
| Rayene   | Page Administrateur, Page de commande, Page de livraison.                                  | Fichier de conception. |

## 3. Problèmes rencontrés et solutions apportées

### 3.1. Problèmes Organisationnels & Humains

- **Problème** : Risque de désynchronisation lors du travail en autonomie sur la structure globale du site.
- **Solution** : Utilisation intensive du tableau Trello pour savoir "qui fait quoi" en temps réel et organisation d'un bilan collectif le samedi pour s'assurer de la cohérence visuelle avant le rendu final.

### 3.2. Problèmes Techniques

#### A. Conflits de configuration Git

- **Problème** : Radouane et Matthieu ne parvenaient pas à "push" leur travail sur GitHub. Le message d'erreur indiquait que Git ne savait pas comment réconcilier les branches (besoin de préciser la stratégie de **pull** : merge ou rebase).
- **Solution** : Radouane et Matthieu ont chacun trouvé la solution en autonomie de leur côté. Ils ont configuré **git config** pour autoriser le merge par défaut, ce qui a permis de débloquer la situation et de synchroniser leurs travaux.

## B. Redondance et maintenance du code CSS

- **Problème** : Lors de la mise en commun, le groupe a constaté que le code CSS de la barre de navigation et du footer était recopié dans chaque fichier. Cela rendait toute modification laborieuse (obligation de modifier chaque fichier un par un).
  - **Solution Temporaire** : Travail prioritaire sur la balise **<main>** en ignorant le reste ou copie temporaire.
  - **Solution Définitive** : Radouane a procédé à une **centralisation du CSS**. Un fichier CSS commun regroupe désormais les variables de couleurs, les styles du **body**, de la **nav**, du **footer** et du **main**. Pour le HTML, la structure a été uniformisée sur chaque page pour cette phase.
- 

## 4. Bilan de la Phase #1

Cette première phase nous a permis de poser des bases solides, tant sur le plan technique qu'organisationnel. L'utilisation d'outils collaboratifs (GitHub, Trello, Discord) a été déterminante pour surmonter les obstacles de configuration initiaux. Le site présente désormais une identité visuelle cohérente et une structure prête à accueillir les fonctionnalités dynamiques des phases suivantes.

## 5. Rapport de Projet - Phase #2

**Période :** Phase #2 (du 16/03 au 05/04 2026)

**Objectif principal :** Passage d'un site statique (HTML/CSS) à un site dynamique (PHP) et gestion des données côté serveur.

### 5.1. Calendrier de réalisation (Planning Phase #2)

La phase 2 s'est concentrée sur la dynamisation de nos maquettes et la mise en place de la logique serveur via PHP.

**Étape 1 : Restructuration de l'architecture.** Transition des fichiers .html vers .php et centralisation des éléments communs. Et **Correction de la phase 1** (arborescence des fichiers, ajout de l'e-mail, correction variable globale, etc).

**Étape 2 : Conception de la base de données :** Dans un premier temps, nous devions utiliser le langage **SQL**, mais pour des soucis de temps (d'apprentissage), nous nous sommes redirigé vers les fichiers **JSON**, beaucoup plus pratiques (fonctions intégrées à php, facilité d'utilisation, ...) et déjà connus pour le groupe.. Définition stricte des champs utilisateurs et création de l'arborescence de stockage des données pour séparer les données clairement (**plus détaillé dans la partie conception ci-dessous**)

**Étape 3 : Développement de la logique métier principale.** Développement en parallèle des grands axes du site :

- *Authentification* : Traitement des formulaires d'inscription et de connexion, hachage sécurisé des mots de passe, et mise en place des variables de Session (`$_SESSION`).
- *E-commerce* : Création et gestion dynamique du panier utilisateur.
- *Administration* : Création de la logique de lecture des fichiers JSON pour les tableaux de bord spécifiques (liste des utilisateurs pour l'Admin, gestion des commandes pour le Restaurateur, et commande en cours pour le Livreur).

**Étape 4 : Intégration avancée et dynamisation des interfaces.** Mise en place du processus de validation des commandes et intégration technique de l'API bancaire (Cy Bank) pour simuler le paiement. Affichage conditionnel des interfaces selon le rôle de l'utilisateur (Client, Admin, etc.) et mise en place des pages de Suivi, d'Histoire et du système de feedback (Avis).

**Étape 5 : Déploiement local et tests.** Utilisation du serveur web intégré de PHP pour le débogage et validation des critères du cahier des charges (ex: création de comptes de test dont 2 administrateurs).

## 5.2. Répartition des tâches (Phase #2)

L'équipe s'est répartie les missions en fonction des affinités avec la logique backend et l'intégration :

**Matthieu** : Gestion complète de l'authentification. Développement des scripts PHP pour *inscription.php*, *connexion.php*, *logout.php*, et gestion dynamique de la page *profil.php* via les Sessions. Implémentation des messages flash (Popup Toast).

**Radouane** : Refonte de l'arborescence globale (fichiers inclus, chemins relatifs). Dynamisation PHP des scripts *Accueil.php* et *Presentation.php*. Création et gestion dynamique du panier, du suivi client et de l'historique des commandes. Implémentation de l'API Cy Bank. Rédaction du présent rapport.

**Rayene** : Développement de la logique de l'admin. Traitement PHP pour *index\_admin.php* (lecture du JSON pour afficher la liste des utilisateurs), et gestion de la page Commandes côté restaurateur et Livraison côté livreur (dashboards). Rédaction du document de conception.

## 5.3. Format des Données (Conception Serveur)

Nous avons opté pour une architecture basée sur des fichiers **JSON**. très simple, léger et natif, c'est parfaitement adapté à la taille de notre projet. Les fonctions php intégrées sont extrêmement pratiques.

Nos fichiers de données sont stockés de manière sécurisée dans un dossier dédié nommé *data/*, situé à la racine du projet (au même niveau que les dossiers *css/* et *assets/*).

### Structure détaillée du fichier *utilisateurs.json* :

Ce fichier stocke un tableau d'objets. Chaque objet représente un utilisateur et contient exactement les clés suivantes (qui correspondent au code de notre fichier d'inscription) :

- **login** (String) : Le pseudonyme unique de l'utilisateur.
- **nom** (String) : Le nom de famille.
- **prenom** (String) : Le prénom.
- **naissance** (String) : La date de naissance au format "AAAA-MM-JJ".
- **adresse** (String) : L'adresse postale complète pour la livraison.
- **tel** (String) : Le numéro de téléphone de contact.
- **infos** (String) : Informations complémentaires (digicode, étage...).
- **email** (String) : L'adresse email (utilisée pour vérifier les doublons lors de l'inscription).
- **password** (String) : Le mot de passe crypté via l'algorithme BCrypt ( en PHP) pour des raisons de sécurité. Les mots de passe ne sont jamais stockés en clair, ils sont cryptés avec **password\_hash()** et **password\_verify()**
- **role** (String) : Définit les droits de l'utilisateur ("client", "admin", "restaurateur", "livreur").

- `date_inscription` (String) : Date et heure générées automatiquement lors de la création du compte au format "JJ/MM/AAAA HH:MM:SS".
- `xp` (Integer) : Les points de fidélité de l'utilisateur (initialisés à 0 par défaut).

### Structure détaillée du fichier *commandes.json* :

Ce fichier stocke un tableau d'objets. Chaque objet représente une transaction complète (une commande passée par un client) et contient exactement les clés suivantes :

- `id` (String) : L'identifiant unique de la transaction (généré aléatoirement en PHP). Il permet d'assurer la traçabilité unique de chaque commande.
- `login_client` (String) : Le pseudonyme de l'utilisateur ayant passé la commande. Cette clé agit comme une clé étrangère (concept SQL) reliant la commande au fichier *utilisateurs.json*.
- `login_livreur` (String ou Null) : Le pseudonyme de l'employé assigné à la livraison. La valeur est à *null* tant que la commande n'a pas été prise en charge par un livreur.
- `date` (String) : La date et l'heure exactes de l'enregistrement de la commande, générées via la fonction PHP `date('Y-m-d H:i:s')` (format standard SQL).
- `type_preparation` (String) : Le choix du client concernant le moment de la préparation (ex: "Dès que possible" ou "plus tard").
- `date_livraison_prevue` (String) : L'horaire attendu par le client (ex: "Dès que possible" ou une heure spécifique).
- `contenu` (Object) : Un dictionnaire de données stockant le panier du client. Les clés correspondent aux identifiants bruts des articles (ex: "plat\_2", "menu\_1") et les valeurs correspondent aux quantités (Integer). Cette structure intelligente évite la redondance et facilite le calcul des totaux.
- `montant` (Float) : Le prix total de la commande en euros (décimal).
- `statut` (String) : L'état d'avancement de la commande au sein de notre flux de travail ("attente\_paiement", "acceptee", "refusee", "preparation", "en\_cours", "livree"). Cette clé est cruciale pour le filtrage de l'affichage sur les tableaux de bord des différents rôles (suivi client, interface restaurateur, interface livreur).

### Structure détaillée du fichier *avis.json* :

Ce fichier stocke un tableau d'objets. Chaque objet représente un retour d'expérience (feedback) laissé par un client suite à une livraison terminée. Il contient les clés suivantes :

- `id_commande` (String) : L'identifiant de la commande concernée. Il permet de lier l'avis à une transaction précise et est utilisé dans notre code pour empêcher un utilisateur de noter deux fois la même commande.
- `login_client` (String) : Le pseudonyme de l'auteur de l'avis.
- `note_livraison` (Integer) : Une valeur numérique (de 1 à 5) évaluant la qualité du déploiement (livraison).
- `note_qualite` (Integer) : Une valeur numérique (de 1 à 5) évaluant la qualité des produits consommés.

- `commentaires` (String) : Un champ de texte libre permettant au client de détailler son expérience. Les données insérées ici sont sécurisées lors de l'affichage avec `htmlspecialchars()` pour contrer les failles XSS.
- `date_avis` (String) : La date et l'heure de la soumission du formulaire, au format "AAAA-MM-JJ HH:MM:SS".

#### Structure détaillée du fichier *plats.json* :

Ce fichier contient un tableau d'objets représentant nos articles individuels (des composants informatiques revisités en plats culinaires). Chaque objet contient les clés suivantes :

- `id` (Integer) : L'identifiant unique du produit.
- `nom` (String) : L'appellation commerciale du produit
- `description` (String) : Le descriptif détaillé du produit, mêlant caractéristiques techniques et saveurs culinaires.
- `catégorie` (String) : La famille du produit (ex: "Carte Graphique"). Cette clé est utilisée pour le système de filtrage et d'affichage ciblé sur la boutique.
- `prix` (Float) : Le tarif unitaire du produit en euros.
- `image` (String) : Le chemin d'accès relatif vers l'image d'illustration (ex: "assets/icones\_presentation/RadeonRX7600.png").
- `alt` (String) : Le texte alternatif décrivant l'image. Cette clé est cruciale pour respecter les normes d'accessibilité du web (W3C) et pour le référencement naturel.
- `allergenes` (Array de Strings) : Une liste des saveurs ou ingrédients notables (ex: ["Chocolat", "Fraise"]).

#### Structure détaillée du fichier *menus.json* :

Ce fichier regroupe nos offres combinées. Il stocke un tableau d'objets structurés de la manière suivante :

- `id` (Integer) : L'identifiant unique du menu.
- `nom` (String) : L'appellation commerciale de la formule (ex: "Menu RTX").
- `description` (String) : Un résumé textuel des éléments qui composent ce menu.
- `prix_total` (Float) : Le prix de vente global du menu.
- `plats_inclus` (Array d'Integers) : Un tableau contenant les identifiants (*id*) des plats individuels constituant le menu. Ces valeurs agissent comme des clés étrangères qui pointent directement vers les objets du fichier *plats.json*.
- `personnes_min` (Integer) : Le nombre de personnes minimum pour prendre le menu.

## 5.4. Problèmes rencontrés et solutions apportées

### A. Environnement de développement et Serveur Local (Matériel/Logiciel)

- **Problème** : Matthieu a rencontré des difficultés techniques avec son ordinateur fixe (corruption de fichiers système) rendant impossible l'installation de WSL ou d'un environnement de développement local classique (de type WAMP/XAMPP).
- **Solution** : Mise en place d'un environnement PHP portable et minimaliste. Matthieu a téléchargé les binaires PHP (format ZIP). À l'aide de l'invite de commande (CMD) placée dans le répertoire du projet, il a utilisé le serveur web interne natif de PHP (via la commande `php -S localhost:8000`), ce qui a permis de contourner le problème système et de garantir la continuité de son travail.

## B. Problèmes de chemins relatifs lors des inclusions PHP

- **Problème** : Afin de ne pas répéter le code de notre barre de navigation, nous l'avons isolé dans un fichier `includes/header.php`. Cependant, lors des premiers tests d'inclusion de ce fichier sur nos différentes pages, nous avons constaté que les images (comme le logo) ou les fichiers CSS ne chargeaient pas toujours. Le problème venait du fait que le chemin relatif changeait selon l'emplacement du fichier qui appelait le header.
- **Solution** : Nous avons restructuré l'arborescence de notre projet de manière stricte. Nous nous sommes mis d'accord pour que tous les fichiers PHP de pages (Accueil, Suivi, Historique, etc.) restent à la racine du projet. Ainsi, les liens relatifs vers `css/style.css` ou `assets/icones/logo.png` inscrits dans le header fonctionnent toujours parfaitement, quelle que soit la page appelée.

# 6. Rapport de Projet - Phase #3

**Période** : Phase #3 (du 04/05 au 17/05)

**Objectif principal** : Dynamisation du front-end avec le langage JavaScript, manipulation du DOM et mise en place des requêtes asynchrones pour modifier les pages sans les recharger.

## 6.1. Calendrier de réalisation (Planning Phase #3)

Cette troisième phase s'est concentrée sur l'amélioration de l'expérience utilisateur (UX) en rendant les interfaces plus interactives et réactives.

- **Étape 1 : Amélioration de l'interface visuelle (Thèmes)**. Mise en place d'un système de changement de charte graphique directement géré côté client, avec persistance des choix de l'utilisateur.



- **Étape 2 : Sécurisation et fluidité des formulaires.** Ajout de vérifications en temps réel sur les champs de saisie (inscription, connexion) avant tout envoi au serveur
- **Étape 3 : Dynamisation de la boutique.** Implémentation des tris et des filtres asynchrones sur le catalogue de produits pour éviter les rechargements de page intempestifs.
- **Étape 4 : Gestion des tableaux de bord (Async).** Ajout des requêtes asynchrones pour les actions de l'administrateur, du restaurateur et du livreur.

## 6.2. Répartition des tâches (Phase #3)

Afin de continuer sur notre dynamique, chaque membre a pris en charge la dynamisation JavaScript des pages qu'il avait intégrées lors des phases précédentes:

- **Radouane :**
  - Mise en place du système de changement de thèmes (Rouge, Rose, Vert) géré par JavaScript et sauvegarde du choix via les cookies.
  - Développement des algorithmes de tri (croissant/décroissant sur les données déjà chargées) et de filtres (via requêtes asynchrones/Fetch) sur la page de présentation des produits.
- **Matthieu :**
  - Dynamisation des formulaires de connexion et d'inscription : vérification des données côté client sans rechargement de page et affichage des messages d'erreur en temps réel.
  - Gestion asynchrone de la modification des informations de l'utilisateur depuis la page Profil.
- **Rayene :**
  - Gestion asynchrone des tableaux de bord : modification du statut des commandes par le restaurateur (passage en préparation, prête, assignation livreur) et par le livreur (livraison effectuée).
  - Correction et amélioration de la partie Administrateur (débutée en Phase 2) : implémentation de la fonction de blocage/déblocage d'un utilisateur de manière 100% asynchrone.

## 6.3. Problèmes rencontrés et solutions apportées

### A. Erreurs de formatage JSON lors des requêtes asynchrones (Fetch)

- Problème : Lors de nos premières requêtes `fetch()` vers nos scripts PHP (pour la gestion des tableaux de bord ou des filtres), la console du navigateur affichait régulièrement l'erreur `SyntaxError: Unexpected token`. Le JavaScript n'arrivait pas à lire les données renvoyées par le serveur.
- Solution : Le problème venait du fait que certains de nos fichiers PHP affichaient du code HTML ou des espaces blancs en plus des données. Nous avons dû nettoyer nos fichiers PHP "API" pour qu'ils ne renvoient strictement que le résultat du `json_encode()` et nous avons ajouté l'en-tête `header('Content-Type: application/json');` pour que la communication entre le serveur et le client soit parfaitement fluide.

## 6.4. Bilan de la Phase #3

L'intégration de JavaScript a transformé notre projet statique en une véritable application web dynamique (SPA - Single Page Application). Le fait d'avoir solidifié notre architecture CSS et notre base de données JSON lors des phases 1 et 2 nous a permis de nous concentrer pleinement sur la logique asynchrone (AJAX/Fetch). Les parcours utilisateurs (de la consultation du catalogue à la livraison) sont désormais fluides, interactifs, et répondent aux standards d'accessibilité et de performance attendus pour la soutenance finale.

## 7. Phase #4

**Période :** Phase #4 (Mai 2026) **Objectif principal :** Sécurisation, évaluation de l'accessibilité, finalisation du site et préparation à la soutenance orale.

### 7.1. Calendrier de réalisation & Sécurisation

Pour cette dernière phase, l'accent a été mis sur la robustesse de l'application et la sécurité absolue des données, conformément aux exigences du cahier des charges qui évalue si des utilisateurs mal intentionnés peuvent perturber le système.

- **Sécurité des formulaires (Double vérification) :** Nous avons implémenté des contrôles stricts via des expressions régulières (Regex) à la fois côté client (JavaScript) et côté serveur (PHP). Par exemple, l'ajout récent et la vérification stricte du code postal (exactement 5 chiffres) et de la ville (uniquement des lettres et tirets) sur les pages d'inscription et de profil empêchent toute saisie incohérente ou tentative d'injection.
- **Accessibilité et UX :** Amélioration des compteurs dynamiques en temps réel (ex: "Il manque X caractères" calculé à chaque frappe clavier). De plus, l'utilisation de l'attribut HTML `novalidate` nous a permis de bloquer les pop-ups natifs intempestifs

des navigateurs, garantissant que nos scripts JS maison aient le contrôle exclusif de l'affichage des erreurs.

- **Prévention des failles** : Sécurisation totale des requêtes asynchrones ([fetch](#)) sur la page de profil. Nous nous sommes assurés qu'un utilisateur malveillant ne puisse pas contourner le JavaScript pour envoyer de mauvaises données (ex: modifier son adresse avec du code) au fichier de réception [update\\_profil.php](#).

## 7.2. Bilan de la Phase #4 et du Projet

Le site web *Silicon Carne* est désormais entièrement fonctionnel et sécurisé. Il couvre avec succès l'intégralité du cycle de vie d'une commande (de la navigation du client à la livraison, en passant par les tableaux de bord interactifs du personnel). La sécurité a été éprouvée contre les mauvaises saisies, l'interface est interactive grâce à JavaScript, et l'architecture JSON est stable et propre